Q23: What Is an Agent Harness — and When the Agent Fails, Which Layer Is at Fault?

DLS 2026 — Module (Paradise)

October 2026

Learning objectives

After this module, a student should be able to (i) name the components of an agent harness and say what each one contributes to the system's behavior, (ii) distinguish a model failure from an instruction failure from a harness failure, (iii) explain why the three layers interact — why a badly designed harness can *induce* model failures — and (iv) design an attribution procedure (logs, replay, ablation) that answers "which layer caused this failure?" with evidence rather than vibes.

1 What a harness is, and why it matters

A frontier LLM, run bare, is a function from a prompt to a completion. Almost nothing that people mean by "agent" lives in that function. The *harness* is everything else: the code that runs before the model sees anything (context assembly, tool schemas, system prompt), the code that runs after the model speaks (tool dispatch, result injection, error formatting), and the code that decides whether the loop continues at all (permission gates, budgets, stop conditions). Every agentic system you have used this semester — Claude Code, Codex CLI, Hermes — is a frontier model wrapped in a few thousand lines of such code [3].

The harness matters because it is the *only* layer most teams can actually change. You cannot retrain the model; you usually cannot rewrite the tool you are calling; you can always rewrite the harness. The practical consequence is that a large fraction of "agent quality engineering" is harness engineering: what goes into the context, how tools are described, how errors are phrased, when to stop. Anthropic's engineering guidance on agents makes the same point from the other direction — the successful agent deployments they observed were built from simple, composable patterns with carefully designed tool interfaces, not from exotic orchestration [3].

Why this course runs on harnesses. You are reading this note inside a harness. When this module was drafted, an agent (this one) received a task description, assembled a context, called tools, and wrote files. When you use Claude Code or Codex, the model in the loop is fixed, but the harness decides what files you see, what tools exist, what the permission prompts say, and what happens on a timeout. A course about LLM agents that never looks at the harness layer is a course about the ghost and not the machine.

2 Anatomy of a harness

Concretely, a minimal agentic harness — the ReAct loop and its descendants [6] — has six parts. Table 1 maps them to what you can observe in a real tool.

Component	What it does	Failure it causes when wrong
Context construction	assembles system prompt, file dumps, history into the model input	context rot: stale or truncated context; the model answers a question the user no longer asks
Tool schemas	declares each tool's name, arguments, semantics	the model hallucinates arguments that the schema never offered, or never calls a tool it needed
Dispatch	validates and routes model calls to real code	silent schema drift: harness accepts yesterday's argument names and forwards them wrong
Error surfaces	formats failures back into the context	opaque errors teach the model to retry blindly instead of fixing the cause
Permission gates	decides which actions need human approval	a too-narrow gate blocks correct plans; a too-wide gate leaks destructive actions
Stopping	max turns, token budgets, completion detection	a timeout kills a run that was about to succeed; a missing stop condition burns a budget forever

Table 1: The six components of a minimal harness, and the characteristic failure each produces when designed wrong.

Two of these deserve emphasis because they are where the model's competence and the harness's design meet. *Tool schemas* are, in effect, an API for the model's benefit: SWE-agent's central result is that an interface designed specifically for the model — the "agent-computer interface" — improved SWE-bench resolve rates dramatically over the human-oriented Linux shell, with the model weights untouched [5]. (The model's own ability to acquire tool use is a separate, trainable capacity — Toolformer showed models can learn to insert tool calls themselves [2] — but a production harness still has to declare the tools.) *Error surfaces* are the harness's teaching channel: the same failure, phrased as "command failed: exit 1" versus "command failed: file not found, did you mean to create it first?", produces completely different model behavior on the next turn. Kim et al.'s RCI agent got large gains on MiniWoB++ largely by prompting the model to criticize and improve its own outputs before acting — i.e., by manufacturing a better error surface than the environment natively provides [1].

3 The core question: attributing failures across three layers

When an agent run fails, there are three suspects, and the differences between them are the point of this module:

- **Model failures**: the model reasoned wrong, hallucinated a tool argument, or selected the wrong tool, *given a correct context and a correct tool result.* The information needed to succeed was in front of it and it failed anyway.

- **Instruction failures**: the model did what it was told; the specification was wrong. An ambiguous prompt, a missing constraint ("never modify files under `venv/`"), a system prompt that omits a tool the task requires. The model is fine; the spec is not.
- **Harness failures**: the machinery corrupted the run. Truncated context, the *wrong* tool result injected into the conversation (a dispatch bug, a caching bug, a race), a timeout that killed a correct run mid-flight, a permission gate that denied a needed action with no explanation.

The taxonomy is easy to state and hard to apply, because the layers interact. The cleanest statement of the interaction: *harness design induces model failures.* If the error surface says only "error", the model learns — in-context, within the run — that errors carry no information, and it retries blindly; the resulting retry storm looks like a model failure (the model keeps doing the same thing) but was caused by the harness. If the tool schema describes an argument ambiguously, the model will pass a plausible-but-wrong value; the harness designer reads this as "the model hallucinates arguments" when the schema was the actual cause. Reflexion-style self-correction [4] works precisely when the feedback signal reaching the model is informative enough to reflect on — which is a property of the harness, not the model.

This is why attribution cannot be done by staring at a transcript. The transcript shows you *what* the model did; it does not show you *what the model was actually given*, which is what the harness controlled.

4 Concrete failure modes

Four failure modes recur across production harnesses; each sits at a different point on the model/harness boundary, which is exactly why they get misattributed.

Context rot. The context accumulates tool output, stale plans, and obsolete file dumps. Nothing is formally "wrong" — every individual injection was correct — but by turn 30 the relevant instruction from turn 2 is buried under 40k tokens of noise, and the model violates it. Looks like: model failure ("it forgot the constraint"). Actually: a context-construction policy with no compaction or prioritization.

Tool-result injection errors. The dispatch layer returns the result of a different call, a stale cached result, or a result truncated to the point of meaninglessness (e.g., a search tool returning only headers). The model then reasons *correctly* from false premises. Looks like: model failure (the conclusion is wrong). Actually: the harness fed it fiction. This is the most insidious class because the model's reasoning on the corrupted input is often flawless — you find the bug only by diffing the injected result against what the tool actually returned.

Retry storms. The model hits an error, retries the identical call, hits it again. Operators read this as "the model is dumb." Frequently the harness's error surface is the cause: an uninformative error gives the model nothing to change, and a per-call timeout that fires just before a slow tool succeeds converts a would-be success into an error+retry cycle. Looks like: model failure. Actually: error-surface and stopping design.

Silent schema drift. A tool's backend gains a renamed field; the harness's schema was not updated; the dispatch layer forwards arguments under old names, and the backend quietly ignores the one that mattered. The model did everything right, the tool "succeeded," and the task still

fails — sometimes much later. Looks like: model failure ("it called the wrong thing") or tool failure. Actually: a schema-validation gap in dispatch.

5 How to instrument attribution

If you cannot attribute by reading transcripts, how do you attribute? Three techniques, in increasing order of cost and conclusiveness. Each corresponds to a project suggestion.

Log tool I/O, not just decisions. Record, for every step, the exact context sent to the model, the raw tool result returned, and the *transformed* tool result injected into the conversation. Injection errors become visible as a diff between the latter two; context rot becomes visible as token counts per section over time. A harness without this logging is unauditable — you cannot even state the hypothesis "the wrong result was injected" without it.

Replay. Given logged contexts, re-run the model on the same context (deterministically, at temperature 0, where feasible). If the replay diverges from the original run at step k, the divergence is either sampling variance or a genuinely nondeterministic component (a live tool); if it reproduces faithfully and still fails, the failure is stable and ablation is the next step. Replay also lets you ask counterfactuals cheaply: re-run step k with the *correct* tool result spliced in — if the run now succeeds, you have proven a harness cause.

Ablation: swap one layer at a time. The decisive instrument. Hold the task fixed and vary exactly one of: the model, the instructions, or the harness version. Three informative cells: (i) same instructions and harness, stronger model — failure persists ⇒ instruction or harness cause; (ii) same model and harness, clarified instructions — failure clears ⇒ instruction cause; (iii) same model and instructions, fixed harness — failure clears ⇒ harness cause. The companion demo (`code/attribution_harness.py`) implements exactly this design on a small task suite: three configurations — good model/good harness, good model/corrupting harness, degraded model/good harness — and prints a table attributing each failure to a layer. Ablation is also the cheapest defense against motivated attribution: teams default to blaming whichever layer they do not own, and a pre-registered swap table forces the evidence to name the layer.

6 The honest summary

We hedge deliberately. The three-layer decomposition is a *model* of failure causation, and real failures are often joint: a harness with poor error surfaces plus a weak model fails where either alone would have survived. Attribution answers "which fix is cheapest and most likely to help," not "which layer sinned." But as a discipline, the sequence — log everything, replay to confirm, ablate to attribute — converts agent debugging from folklore into measurement, and the SWE-agent result stands as the existence proof that harness-layer changes can move agent performance by margins that look like model-generation jumps [5].

References

[1] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. Language models can solve computer tasks. In *Advances in Neural Information Processing Systems*, 2023.

[2] Timo Schick, Jane Dwivedi-Yu, Roberto Dessè, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Patrick Lewis, and Cordelia Schmid. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 2023.

[3] Erik Schluntz and Barry Zhang. Building effective agents. Anthropic Engineering Blog, 2024.

[4] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

[5] John Yang, Carlos E. Jiménez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.

[6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.